Control/Conditional Structures I *(Selection)*

Engr.Tehseen Ahsan

EC-110: Computing Fundamentals (CF)

Assistant Professor, Computer Engineering Department

HITEC University Taxila Cantt, Pakistan

Control Structures

- A computer can process a program:
  - In *sequence*
  - *Selectively* (branch) - making a choice
  - *Repetitively* (iteratively) - looping
- Some statements are executed only if certain conditions are met.
- A condition is met if it evaluates to `true`

Control Structures (continued)

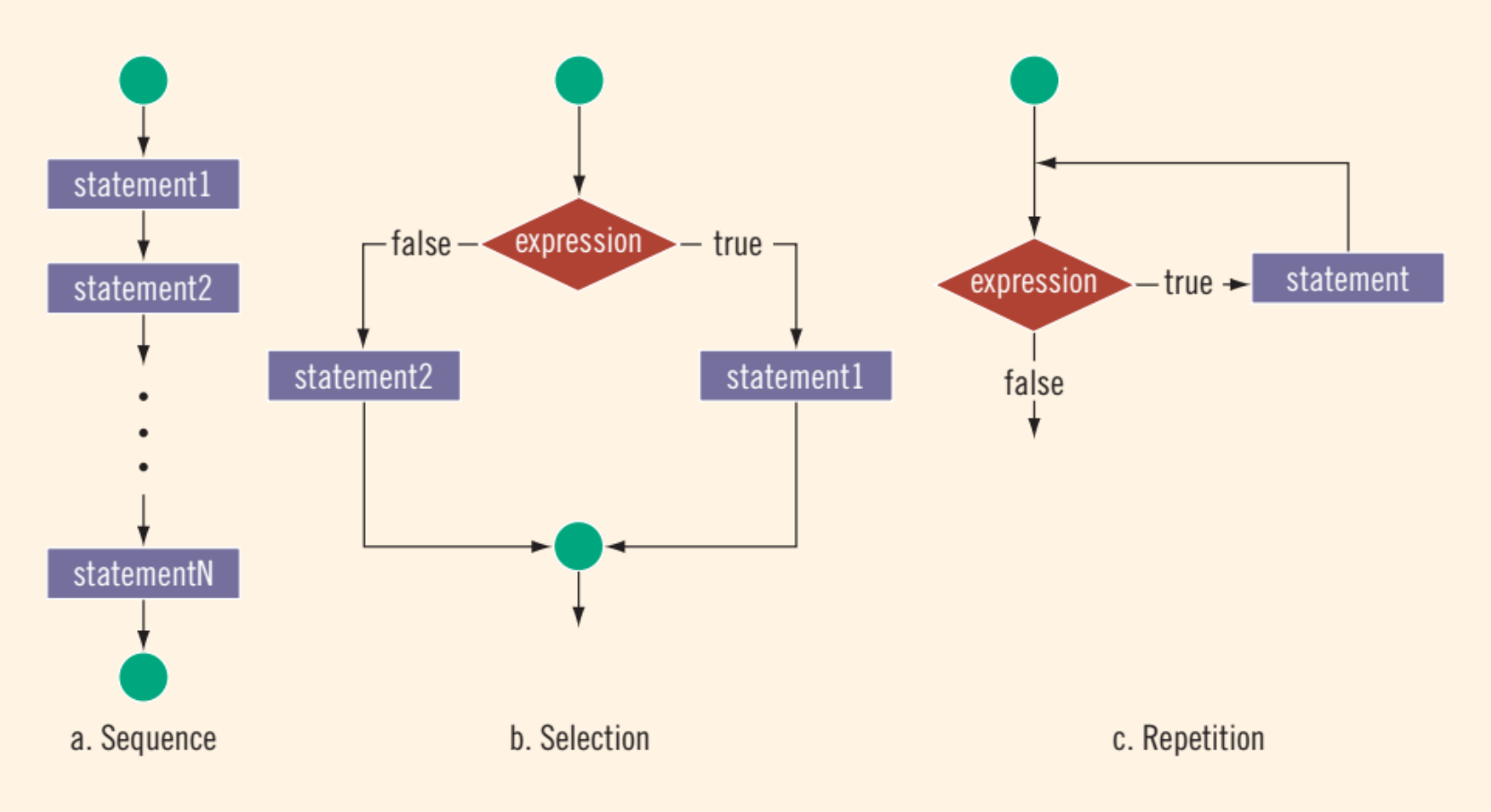


Figure 1

Relational Operators

- A condition is represented by a logical (Boolean) expression that can be `true` or `false`
- *Relational operators*:
  - *Allow comparisons*
  - Require two operands (binary)
  - Evaluate to `true` or `false`

Relational Operators (continued)

Operator	Description
==	equal to
!=	not equal to
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to

Table 1

Relational Operators and Simple Data Types

- You can use the relational operators with all three simple data types:
  - `8 < 15` evaluates to `true`
  - `6 != 6` evaluates to `false`
  - `2.5 > 5.8` evaluates to `false`
  - `5.9 <= 7.5` evaluates to `true`

Comparing Characters

Comparing Characters

For `char` values, whether an expression using relational operators evaluates to `true` or `false` depends on a machine's collating sequence. The collating sequence of some of the characters is:

ASCII Value	Char	ASCII Value	Char	ASCII Value	Char	ASCII Value	Char
32	' '	61	=	81	Q	105	i
33	!	62	>	82	R	106	j
34	"	65	A	83	S	107	k
42	*	66	B	84	T	108	l
43	+	67	C	85	U	109	m
45	-	68	D	86	V	110	n
47	/	69	E	87	W	111	o
48	0	70	F	88	X	112	p
49	1	71	G	89	Y	113	q
50	2	72	H	90	Z	114	r
51	3	73	I	97	a	115	s
52	4	74	J	98	b	116	t
53	5	75	K	99	c	117	u
54	6	76	L	100	d	118	v
55	7	77	M	101	e	119	w
56	8	78	N	102	f	120	x
57	9	79	O	103	g	121	y
60	<	80	P	104	h	122	z

Table 2

Comparing Characters (continued)

Expression	Value of Expression	Explanation
`' ' < 'a'`	`true`	The ASCII value of `' '` is 32, and the ASCII value of `'a'` is 97. Because `32 < 97` is `true`, it follows that `' ' < 'a'` is `true`.
`'R' > 'T'`	`false`	The ASCII value of `'R'` is 82, and the ASCII value of `'T'` is 84. Because `82 > 84` is `false`, it follows that `'R' > 'T'` is `false`.
`'+' < '*'`	`false`	The ASCII value of `'+'` is 43, and the ASCII value of `'*'` is 42. Because `43 < 42` is `false`, it follows that `'+' < '*'` is `false`.
`'6' <= '>'`	`true`	The ASCII value of `'6'` is 54, and the ASCII value of `'>'` is 62. Because `54 <= 62` is `true`, it follows that `'6' <= '>'` is `true`.

Table 3

Logical (Boolean) Operators and Logical Expressions

- Logical (Boolean) operators enable you to combine logical expressions. C++ has three logical (Boolean) operators, as shown in table give below:

Operator	Description
!	not
&&	and
||	or

Table 4

- The *operator* ***!*** *is unary, so it has only one operand*.
- The *operators* ***&&*** *and* **||** *are binary operators i.e., two operands*.

Logical (Boolean) Operators and Logical Expressions (continued)

- ***The ! (NOT) Operator-*** When you use the **!** Operator, **!**True is false and **!**false is true.
- Putting **!** Infront of a logical expression reverses the value of that logical expression.

Expression	!(Expression)
true (nonzero)	false (0)
false (0)	true (1)

Table 5

Logical (Boolean) Operators and Logical Expressions (continued)

- ***The && (AND) Operator-*** From this table, it follows that **Expression1 && Expression2** is True if and only if both **Expression1** and **Expression2** are True; otherwise, **Expression1 && Expression2** evaluates to

Expression1	Expression2	Expression1 && Expression2
true (nonzero)	true (nonzero)	true (1)
true (nonzero)	false (0)	false (0)
false (0)	true (nonzero)	false (0)
false (0)	false (0)	false (0)

Table 6

Logical (Boolean) Operators and Logical Expressions (continued)

EXAMPLE 4-4

Expression	Value	Explanation
`(14 >= 5) && ('A' < 'B')`	`true`	Because `(14 >= 5)` is `true`, `('A' < 'B')` is `true`, and `true && true` is `true`, the expression evaluates to `true`.
`(24 >= 35) && ('A' < 'B')`	`false`	Because `(24 >= 35)` is `false`, `('A' <'B')` is `true`, and `false && true` is `false`, the expression evaluates to `false`.

Logical (Boolean) Operators and Logical Expressions (continued)

- ***The || (OR) Operator-*** Table shown below follows **Expression1 || Expression2** is true if and only if at least one of the expressions, **Expression1** or **Expression2** is true; otherwise, **Expression1 || Expression2** evaluates to False.

Expression1	Expression2	Expression1 || Expression2
true (nonzero)	true (nonzero)	true (1)
true (nonzero)	false (0)	true (1)
false (0)	true (nonzero)	true (1)
false (0)	false (0)	false (0)

Table 7

Logical (Boolean) Operators and Logical Expressions (continued)

EXAMPLE 4-5

Expression	Value	Explanation
`(14>=5) || ('A'>'B')`	`true`	Because `(14 >= 5)` is `true`, `('A' > 'B')` is `false`, and `true || false` is `true`, the expression evaluates to `true`.
`(24>=35) || ('A'>'B')`	`false`	Because `(24 >= 35)` is `false`, `('A' > 'B')` is `false`, and `false || false` is `false`, the expression evaluates to `false`.
`('A'<='a') || (7 !=7)`	`true`	Because `('A' <= 'a')` is `true`, `(7 != 7)` is `false`, and `true || false` is `true`, the expression evaluates to `true`.

Selection: **if** and **if**…**else**

- Although there are only two logical values, ***true*** and ***false***, they turn out to be extremely useful because they permit programs to incorporate decision making that alters the processing flow.
- In C++, *there are two selections, or branch control structures*: ***if*** statements and the ***switch structure.***

Selection: **if** and **if**…**else** (continued…)

One-Way Selection

- A bank would like to send a notice to a customer if his/her checking account balance falls below the required minimum balance. That is, if the account balance is below the required minimum balance, it should send a notice to the customer; otherwise, it should do nothing.
- The syntax of one-way selection is:

```
if (expression)
    statement
```

Selection: **if** and **if**…**else** (continued…)

One-Way Selection

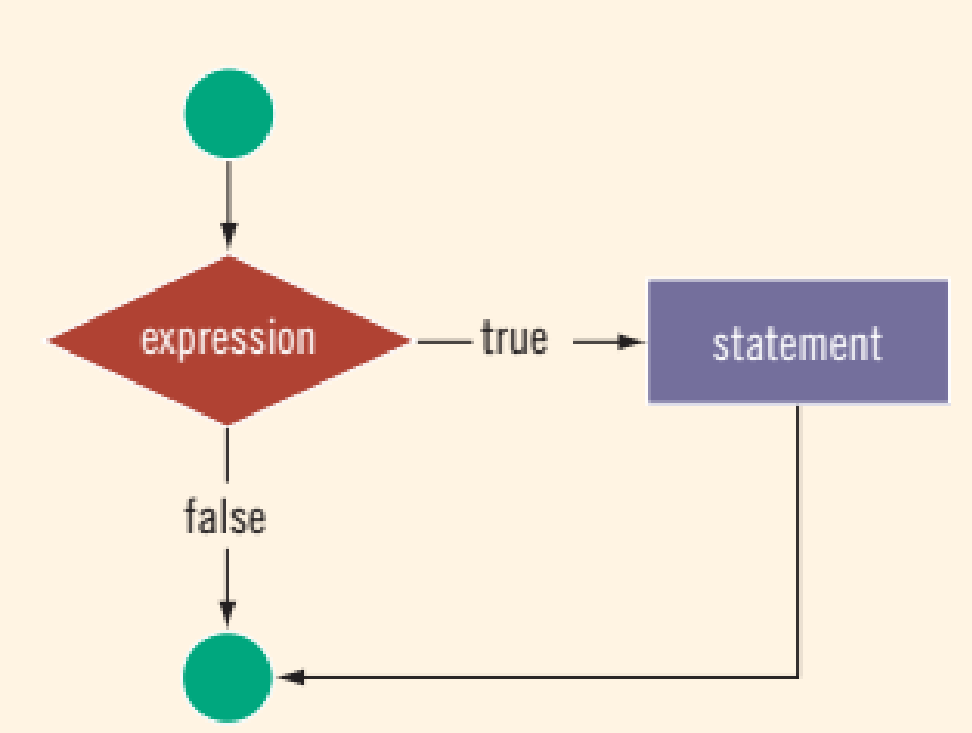


Figure 2

Selection: **if** and **if**…**else** (continued…)

EXAMPLE 4-8

The following C++ program finds the absolute value of an integer.

```
//Program: Absolute value of an integer

#include <iostream>

using namespace std;

int main()
{
    int number, temp;

    cout << "Line 1: Enter an integer: ";          //Line 1
    cin >> number;                                 //Line 2
    cout << endl;                                  //Line 3
```

Selection: **if** and **if**…**else** (continued…)

```
    temp = number;                                          //Line 4

    if (number < 0)                                         //Line 5
        number = -number;                                   //Line 6

    cout << "Line 7: The absolute value of "
         << temp << " is " << number << endl;               //Line 7

    return 0;
}
```

Sample Run: In this sample run, the user input is shaded.

```
Line 1: Enter an integer: -6734
Line 7: The absolute value of -6734 is 6734
```

Home Assignment 1

Program 1: (Use of if Statement)

Write a program to compute and output the penalty on an unpaid credit card balance. The program assumes that the interest interest rate on the unpaid balance is 1.5% per month.

Selection: **if** and **if**…**else** (continued…)

Two-Way Selection

- There are many programming situations in which you must choose between two alternatives. For example, if a part-time employee works overtime, the paycheck is calculated using the overtime payment formula; otherwise, the paycheck is calculated using the regular formula.
- To implement two-way selections- C++ provides the ***if… else*** statement. Two-way selection uses the following syntax:

```
if (expression)
    statement1
else
    statement2
```

Selection: **if** and **if**…**else** (continued…)

Two-Way Selection

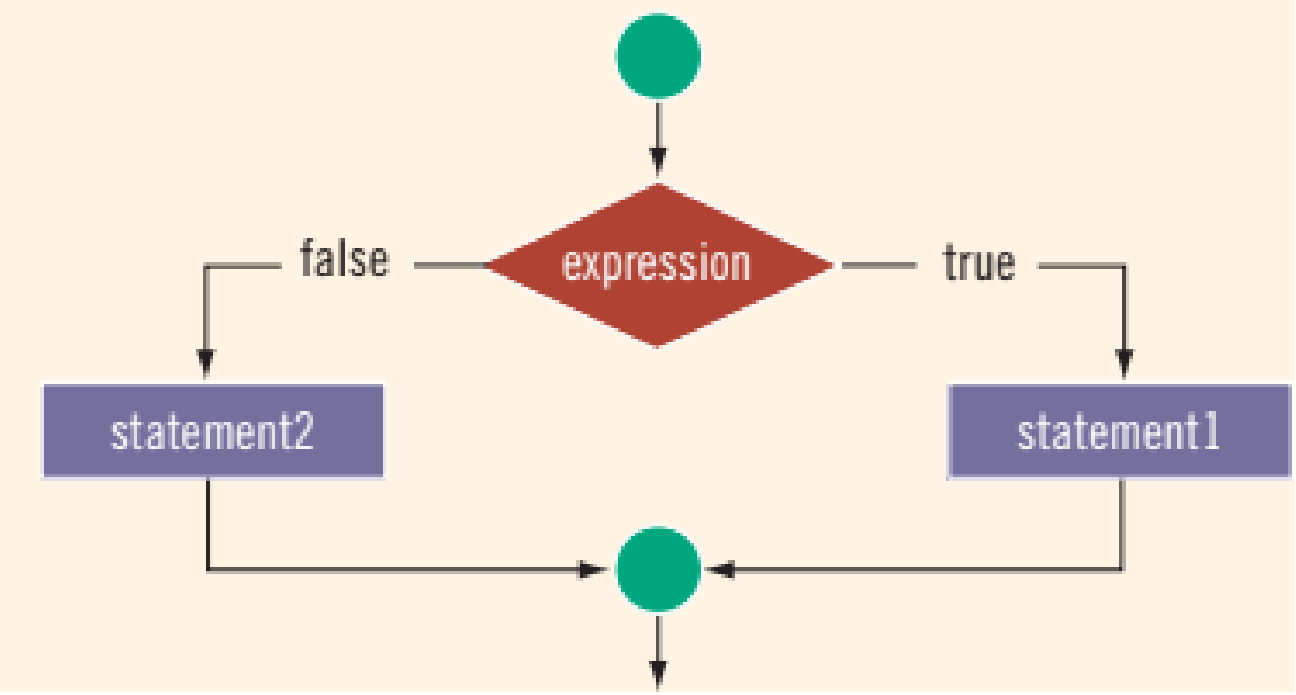


Figure 3

Selection: **if** and **if…else** (continued…)

Program: *Write a program that inputs a number and finds whether it is even or odd using if-else structure.*

```
#include<iostream>
using namespace std;

int main()
{
int n;
cout<<"Enter a number:";
cin>>n;
if(n%2==0)
            cout<<n<<" is even. ";
            else
            cout<<n<<" is odd. ";
            return 0;
}
```

OUTPUT

```
Enter a number:63
63 is odd.
--------------------------------
```

```
Enter a number:26
26 is even.
--------------------------------
```

Home Assignment 1

Program 2: ***(Use of if and if…else Statement)***

Write a program to check whether an integer is positive or negative. Consider Zero as a positive number.

Compound (Block of) Statements

- The ***if*** and ***if…else*** structures control only one statement at a time. Suppose, however, that you want to execute more than one statement if the expression in an ***if*** or ***if…else*** statement evaluates to ***true***.
- To permit more complex statements, C++ provides a structure called a ***compound statement*** or a ***block of statements***.
- A compound statement uses the following syntax:

```
{
    statement_1
    statement_2
       .
       .
       .
    statement_n
}
```

Compound (Block of) Statements (continued…)

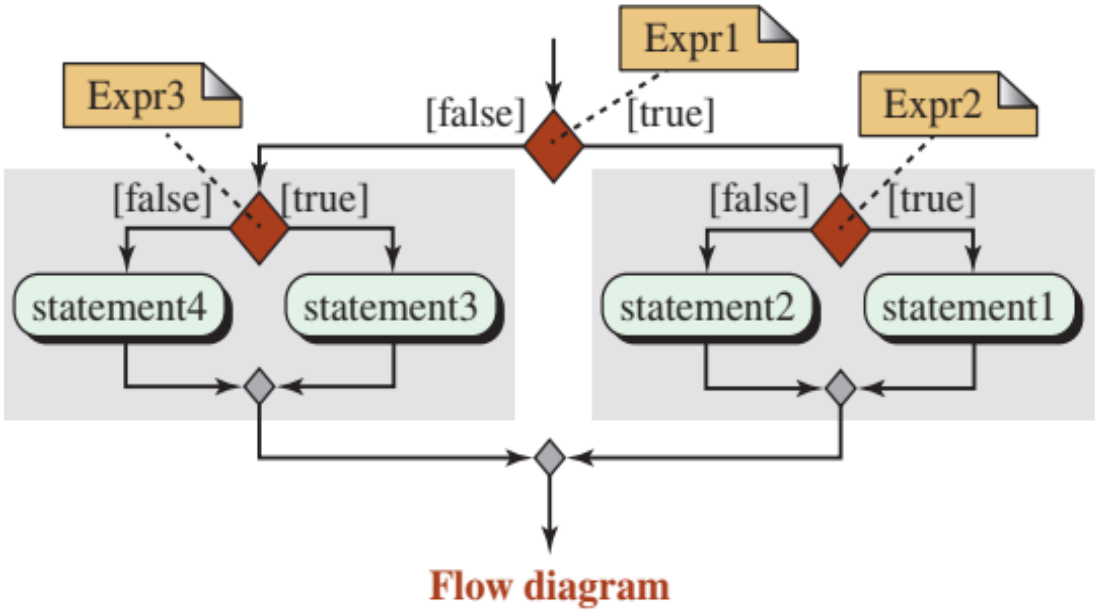
- Thus, instead of having a simple two-way selection similar to following code:

```
if (age >= 18)
    cout << "Eligible to vote." << endl;
else
    cout << "Not eligible to vote." << endl;
```

you could include compound statements, similar to the following code:

```
if (age >= 18)
{
    cout << "Eligible to vote." << endl;
    cout << "No longer a minor." << endl;
}
else
{
    cout << "Not eligible to vote." << endl;
    cout << "Still a minor." << endl;
}
```

Nested if…else Statement



Flow diagram

```
if (expr1)
{
    if (expr2)
    {
        statement1;
    }
    else
    {
        statement2;
    }
}
else
{
    if (expr3)
    {
        statement3;
    }
    else
    {
        statement4;
    }
}
```

Code

Figure 4

Nested if…else Statement (continued…)

Program 4.5 Finding the relationship between two numbers

```
1  /*************************************************************
2   * Find if a number is greater than, equal, or less than another *
3   *************************************************************/
4  #include <iostream>
5  using namespace std;
6
7  int main ()
8  {
9    // Declaration
10    int num1, num2;
11   // Get input values
12    cout << "Enter the first number: ";
13    cin >> num1;
14    cout << "Enter the second number: ";
15    cin >> num2;
```

Nested if…else Statement (continued…)

```
16  // Decision using nested if-else statement
17  if (num1 >= num2)
18  {
19      if (num1 > num2)
20      {
21          cout << num1 << " > " << num2;
22      }
23      else
24      {
25          cout << num1 << " == " << num2;
26      }
27  }
28  else
29  {
30      cout << num1 << " < " << num2;
31  }
32  return 0;
33  }
```

Nested if…else Statement (continued…)

```
Run:
Enter the first number: 42
Enter the second number: 32
42 > 32
```

```
Run
Enter the first number: 12
Enter the second number: 12
12 == 12
```

```
Run
Enter the first number: 12
Enter the second number: 28
12 < 28
```

Nested if…else Statement (continued…)

Comparing *if…else* statements with a series of *if* statements

If…else statements	*Series of if statements*

```
if (month == 1)                    //Line 1
    cout << "January" << endl;     //Line 2
else if (month == 2)               //Line 3
    cout << "February" << endl;    //Line 4
else if (month == 3)               //Line 5
    cout << "March" << endl;       //Line 6
else if (month == 4)               //Line 7
    cout << "April" << endl;       //Line 8

else if (month == 5)               //Line 9
    cout << "May" << endl;         //Line 10
else if (month == 6)               //Line 11
    cout << "June" << endl;        //Line 12
```

Program segment (a)

```
if (month == 1)
    cout << "January" << endl;
if (month == 2)
    cout << "February" << endl;
if (month == 3)
    cout << "March" << endl;
if (month == 4)
    cout << "April" << endl;
if (month == 5)
    cout << "May" << endl;
if (month == 6)
    cout << "June" << endl;
```

Program segment (b)

Table 12

Nested if…else Statement (continued…)

Program segment (a) is written as a sequence of `if...else` statements; program segment (b) is written as a series of `if` statements. Both program segments accomplish the same thing. If `month` is `3`, then both program segments output `March`. If `month` is `1`, then in program segment (a), the expression in the `if` statement in Line 1 evaluates to `true`. The statement (in Line 2) associated with this `if` then executes; the rest of the structure, which is the `else` of this `if` statement, is skipped; and the remaining `if` statements are not evaluated. In program segment (b), the computer has to evaluate the expression in each `if` statement because there is no `else` statement. As a consequence, program segment (b) executes more slowly than does program segment (a).

Home Assignment 1

Program 3: ***(Use of Nested If…else Statement)***

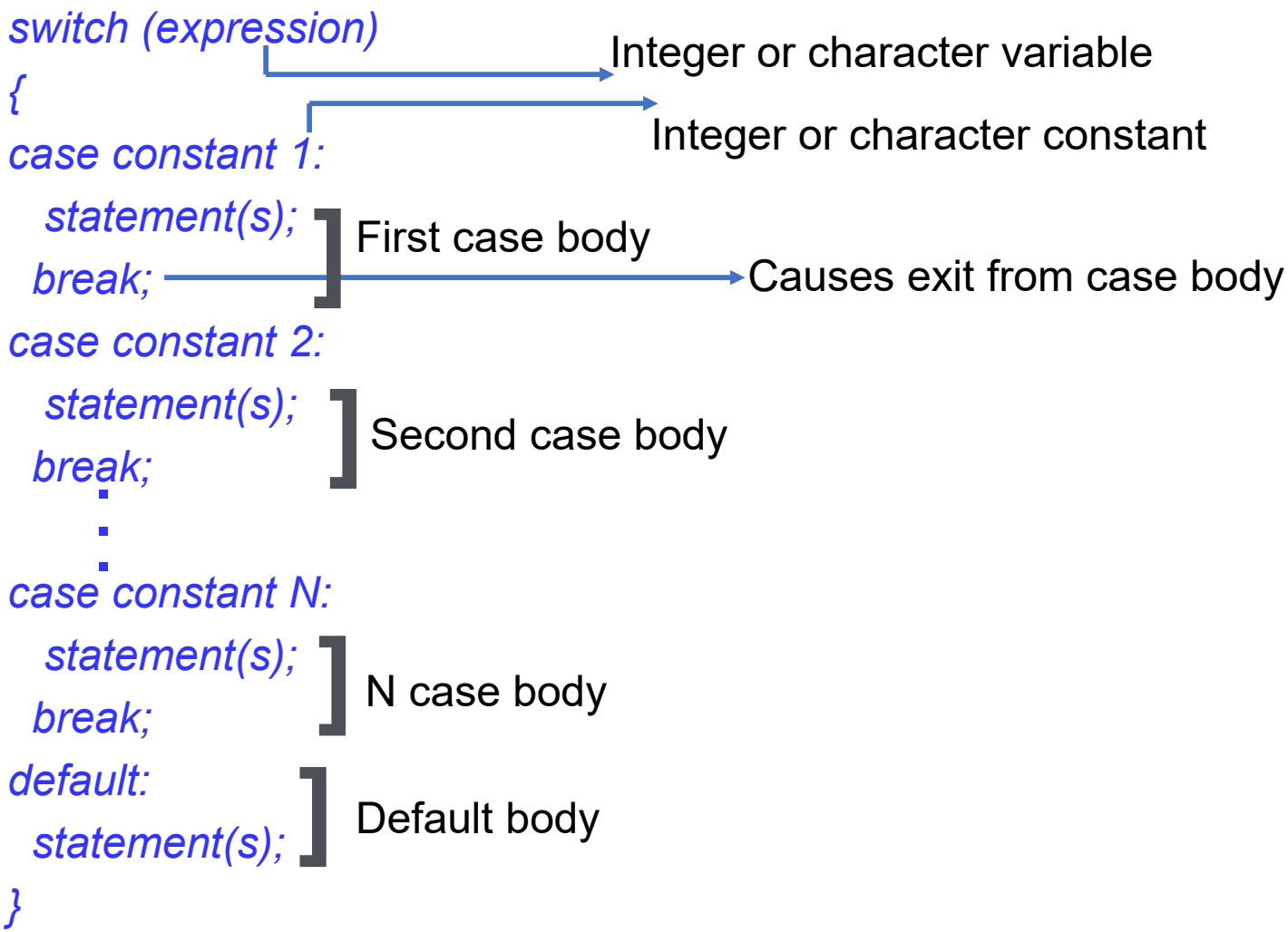
Write a program to read a character to check whether it is a VOWEL or a CONSTANT.

Switch and Break Statements

- *The **switch** statement is another conditional structure. It is a good alternative of **nested if-else**. It can be used easily when there are many choices available and only one should be executed*. Nested if becomes very difficult in such situation.
- ***Working of 'switch' Structure***
  - **Switch** statement compares the result of a single expression with multiple cases. The expression is evaluated at the top of switch statement and its result is compared with different cases. Each case represents one choice. If the result matches with any case the corresponding block of statements is executed.
  - The **default** label appears at the end of all **case** blocks. It is executed only when the result of **expression** does not match with any case label.
  - When the result of the expression matches with a **case** block, the corresponding statements are executed. The **break** statement comes after these statements and the control exits from **switch** body. If **break** is not used, all case blocks that come after the matching case, will also be executed.

Switch and Break Statements (continued…)

Syntax of switch and break statements



Switch and Break Statements (continued…)

Program: *Write a program that inputs number of day of week and displays the name of the day. For example if user enters 1, it displays "Friday" and so on.*

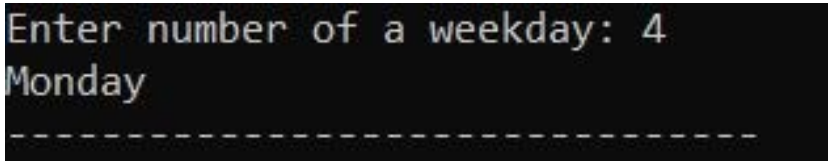
```
#include<iostream>
using namespace std;

int main()
{
int n;
cout<<"Enter number of a weekday: ";
cin>>n;
switch(n)
{
case 1:
cout<<"Friday";
break;
case 2:
cout<<"Saturday";
break;
```

Switch and Break Statements (continued…)

```
case 3:
cout<<"Sunday";
break;
case 4:
cout<<"Monday";
break;
case 5:
cout<<"Tuesday";
break;
case 6:
cout<<"Wednesday";
break;
case 7:
cout<<"Thursday";
break;
default:
cout<<"Invalid Number";
}
        return 0;
}
```

OUTPUT



Conditional Operator

- Conditional operator is a *decision-making structure*. *It can be used in place of simple if-else structure*. It is called **ternary operator** as it uses three operands.
- ***Syntax***
  - The syntax of conditional operator is as follows:

```
(condition ? true-case statement: false-case statement);
```

condition: the condition is specified as relational or logical expression. The condition is evaluated to **true** or **false**.

true-case: It is executed if expression evaluates to **true**.

false-case: It is executed if expression evaluates to **false**.

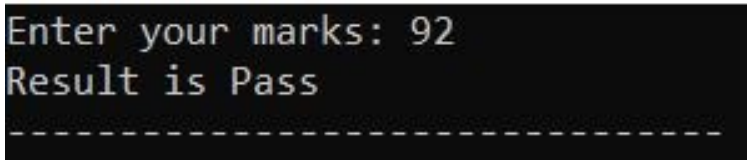
Conditional Operator (continued…)

Program: *Write a program that inputs marks of a student and displays "Pass" if marks are greater than 40 and "Fail" otherwise by using conditional operator.*

```
#include<iostream>
using namespace std;

int main()
{
int marks;
cout<<"Enter your marks: ";
cin>>marks;
cout<<"Result is " <<(marks>40 ? "Pass" : "Fail");
return 0;
}
```

OUTPUT



‘goto’ Statement

- *The ‘goto’ statement is used to move the control directly to a particular location of the program by using label.* A label is a name given to a particular line of the program. A label is created with a valid identifier followed by a colon(:).
- ***Syntax***

```
goto Label;
```

The ‘Label’ indicates the label to which the control is transferred.

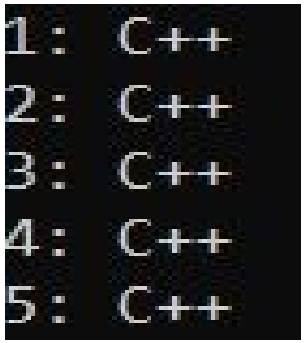
‘goto’ Statement (continued…)

Program: *Write a program that displays “C++” five times using goto statement.*

```
#include<iostream>
using namespace std;

int main()
{
        int n=1;
        loop:
        cout<<n<<": C++"<<endl;
        n++;
        if(n<=5) goto loop;
return 0;
}
```

OUTPUT



References: Textbooks

1. *"C++ Programming: From Problem Analysis to Program Design",6/ed by D.S. Malik, (2013), CENGAGE Learning.*
2. *"Object Oriented Programming in C++," 4/ed by Robert Lafore (2001) .*
3. *"C++ How to Program," 5/ed by Deitel & Deitel (2005).*
4. *"Program with C++ Object Oriented Programming Aikman Series", by C.M Aslam and T.A Qureshi.*
5. *Related documents from open source, mainly internet.*